Think OS

A Brief Introduction to Operating Systems

Version 0.7.4

Think OS

A Brief Introduction to Operating Systems

Version 0.7.4

Allen B. Downey

Green Tea Press

Needham, Massachusetts

Copyright © 2015 Allen B. Downey.

Green Tea Press
9 Washburn Ave
Needham MA 02492

Permission is granted to copy, distribute, and/or modify this document under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License, which is available at http://creativecommons.org/licenses/by-nc-sa/4.0/.

The LaTeX source for this book is available from http://greenteapress.com/thinkos.

Preface

In many computer science programs, Operating Systems is an advanced topic. By the time students take it, they know how to program in C, and they have probably taken a class in Computer Architecture. Usually the goal of the class is to expose students to the design and implementation of operating systems, with the implied assumption that some of them will do research in this area, or write part of an OS.

This book is intended for a different audience, and it has different goals. I developed it for a class at Olin College called Software Systems.

Most students taking this class learned to program in Python, so one of the goals is to help them learn C. For that part of the class, I use Griffiths and Griffiths, *Head First C*, from O'Reilly Media. This book is meant to complement that one.

Few of my students will ever write an operating system, but many of them will write low-level applications in C or work on embedded systems. My class includes material from operating systems, networks, databases, and embedded systems, but it emphasizes the topics programmers need to know.

This book does not assume that you have studied Computer Architecture. As we go along, I will explain what we need.

If this book is successful, it should give you a better understanding of what is happening when programs run, and what you can do to make them run better and faster.

Chapter 1 explains some of the differences between compiled and interpreted languages, with some insight into how compilers work. Recommended reading: *Head First C* Chapter 1.

Chapter 2 explains how the operating system uses processes to protect running programs from interfering with each other.

Chapter 3 explains virtual memory and address translation. Recommended reading: *Head First C* Chapter 2.

Chapter 4 is about file systems and data streams. Recommended reading: *Head First C* Chapter 3.

Chapter 5 describes how numbers, letters, and other values are encoded, and presents the bitwise operators.

Chapter 6 explains how to use dynamic memory management, and how it works. Recommended reading: *Head First C* Chapter 6.

Chapter 7 is about caching and the memory hierarchy.

Chapter 8 is about multitasking and scheduling.

Chapter 9 is about POSIX threads and mutexes. Recommended reading: *Head First C* Chapter 12 and *Little Book of Semaphores* Chapters 1 and 2.

Chapter 10 is about POSIX condition variables and the producer/consumer problem. Recommended reading: *Little Book of Semaphores* Chapters 3 and 4.

Chapter 11 is about using POSIX semaphores and implementing semaphores in C.

A note on this draft

The current version of this book is an early draft. While I am working on the text, I have not yet included the figures. So there are a few places where, I'm sure, the explanation will be greatly improved when the figures are ready.

0.1 Using the code

Example code for this book is available from `https://github.com/AllenDowney/ThinkOS`. Git is a version control system that allows you to keep track of the files that make up a project. A collection of files under Git's control is called a **repository**. GitHub is a hosting service that provides storage for Git repositories and a convenient web interface.

The GitHub homepage for my repository provides several ways to work with the code:

- You can create a copy of my repository on GitHub by pressing the Fork button. If you don't already have a GitHub account, you'll need to create one. After forking, you'll have your own repository on GitHub

that you can use to keep track of code you write while working on this book. Then you can clone the repo, which means that you copy the files to your computer.

- Or you could clone my repository. You don't need a GitHub account to do this, but you won't be able to write your changes back to GitHub.
- If you don't want to use Git at all, you can download the files in a Zip file using the button in the lower-right corner of the GitHub page.

Contributor List

If you have a suggestion or correction, please send email to `downey@allendowney.com`. If I make a change based on your feedback, I will add you to the contributor list (unless you ask to be omitted).

If you include at least part of the sentence the error appears in, that makes it easy for me to search. Page and section numbers are fine, too, but not quite as easy to work with. Thanks!

- I am grateful to the students in Software Systems at Olin College, who tested an early draft of this book in Spring 2014. They corrected many errors and made many helpful suggestions. I appreciate their pioneering spirit!
- James P Giannoules spotted a copy-and-paste error.
- Andy Engle knows the difference between GB and GiB.
- Aashish Karki noted some broken syntax.

Other people who found typos and errors include Jim Tyson, Donald Robertson, Jeremy Vermast, Yuzhong Huang, Ian Hill.

Contents

Preface **v**

0.1 Using the code vi

1 Compilation **1**

1.1 Compiled and interpreted languages 1

1.2 Static types 1

1.3 The compilation process 3

1.4 Object code 4

1.5 Assembly code 5

1.6 Preprocessing 6

1.7 Understanding errors 6

2 Processes **9**

2.1 Abstraction and virtualization 9

2.2 Isolation 10

2.3 UNIX processes 12

3 Virtual memory **15**

3.1 A bit of information theory 15

3.2 Memory and storage 16

3.3 Address spaces 16

3.4 Memory segments . . . 17
3.5 Static local variables . . . 20
3.6 Address translation . . . 20

4 Files and file systems **23**
4.1 Disk performance . . . 25
4.2 Disk metadata . . . 27
4.3 Block allocation . . . 28
4.4 Everything is a file? . . . 28

5 More bits and bytes **31**
5.1 Representing integers . . . 31
5.2 Bitwise operators . . . 32
5.3 Representing floating-point numbers . . . 33
5.4 Unions and memory errors . . . 35
5.5 Representing strings . . . 36

6 Memory management **39**
6.1 Memory errors . . . 39
6.2 Memory leaks . . . 41
6.3 Implementation . . . 43

7 Caching **45**
7.1 How programs run . . . 45
7.2 Cache performance . . . 47
7.3 Locality . . . 47
7.4 Measuring cache performance . . . 48
7.5 Programming for cache performance . . . 51
7.6 The memory hierarchy . . . 52
7.7 Caching policy . . . 53
7.8 Paging . . . 54